



# Amazon EC2 Auto Scaling Groups (ASG)

## Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

# Introduction to Auto Scaling Groups



An Auto Scaling Group (**ASG**) contains a collection of EC2 instances that are treated as a group for the purposes of automatic scaling and management.

Automatic scaling can occur via:

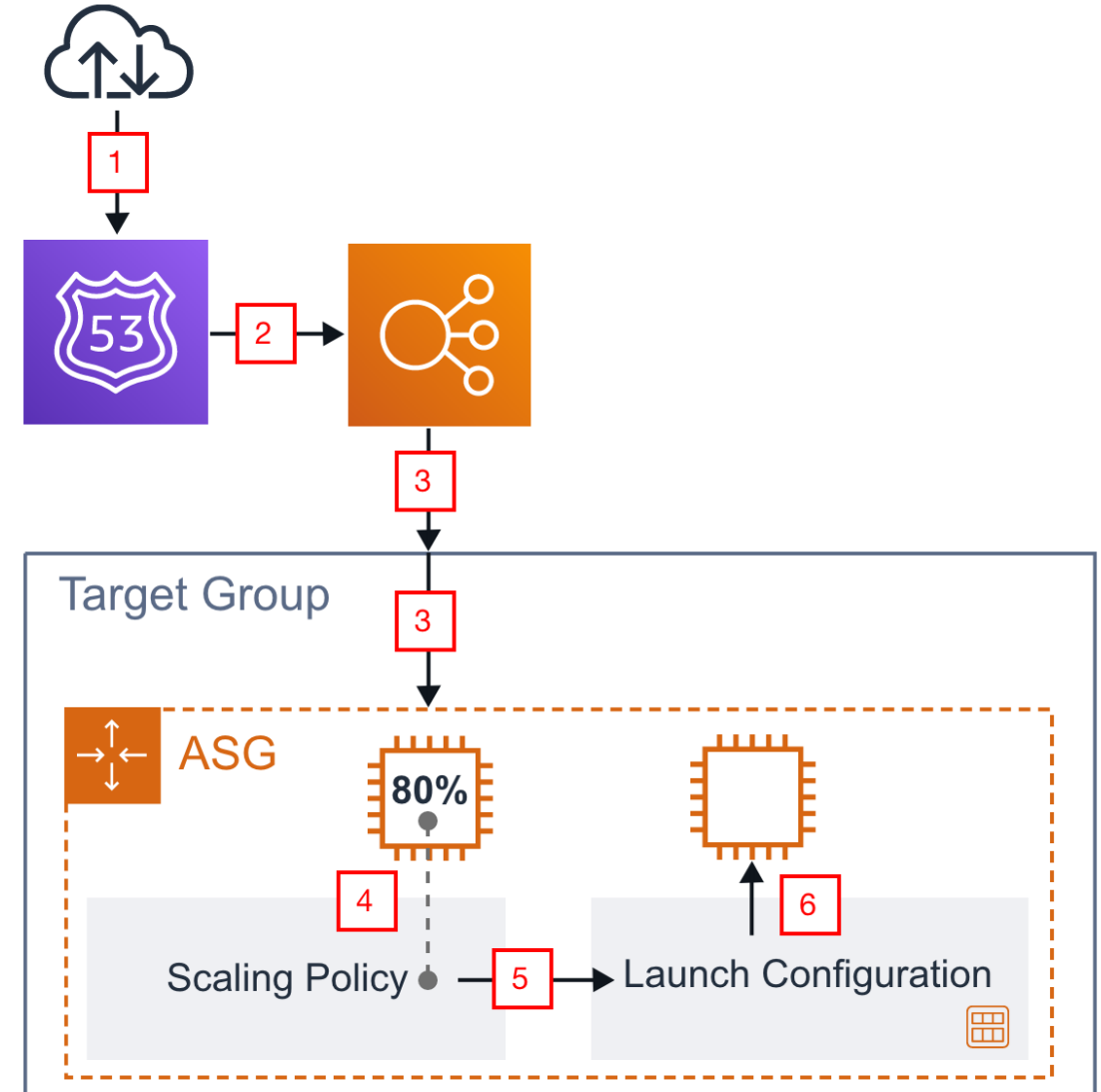
- **Capacity Settings** — set the expected range of capacity
  - Manual Scaling
- **Health Check Replacements** — replace instances if they are determined unhealthy
  - EC2 or ELB Health Checks
- **Scaling Policies** — set complex rules to determine when to scale up or down
  - Simple Scaling
  - Step Scaling
  - Target Tracking Scaling
  - Predictive Scaling

ASGs are used to scale EC2 instances. ECS with EC2 will work, EKS with EC2 will work. Fargate **does not use** ASGs



# ASG - Use Case

1. Burst of traffic from the internet hits our domain.
2. Route53 points that traffic to our load balancer.
3. Our load balancer passes the traffic to its target group.
4. The target group is associated with our ASG and sends the traffic to instances registered with our ASG
5. The ASG Scaling Policy will check if our instances are near capacity.
6. The Scaling Policy determines we need another instance, and it launches a new EC2 instance with the associated Launch Configuration to our ASG



# ASG - Capacity Settings

The size of an Auto Scaling Group is based on:  
**Min Size**, **Max Size**, and **Desired Capacity**.

- **Min Size** is how many EC2 instances should at least be running.
- **Max Size** is the number of EC2 instances allowed to be running.
- **Desired Capacity** is how many EC2 instances you want to ideally run.



```
aws autoscaling update-auto-scaling-group \  
--auto-scaling-group-name my-asg \  
--min-size 2 \  
--max-size 10
```



```
aws autoscaling set-desired-capacity \  
--auto-scaling-group-name my-asg \  
--desired-capacity 2 \  
--honor-cooldown
```

ASG will always launch instances to meet **minimum size capacity**.

Changing the min, max and desired capacity is considered “manual scaling”, since you have to manually change these numbers to change capacity.

# ASG - Health Check Replacements

**Health Check Replacement** is when an ASG replaces an instance if the instance is considered unhealthy. There are two types of health checks ASG can perform

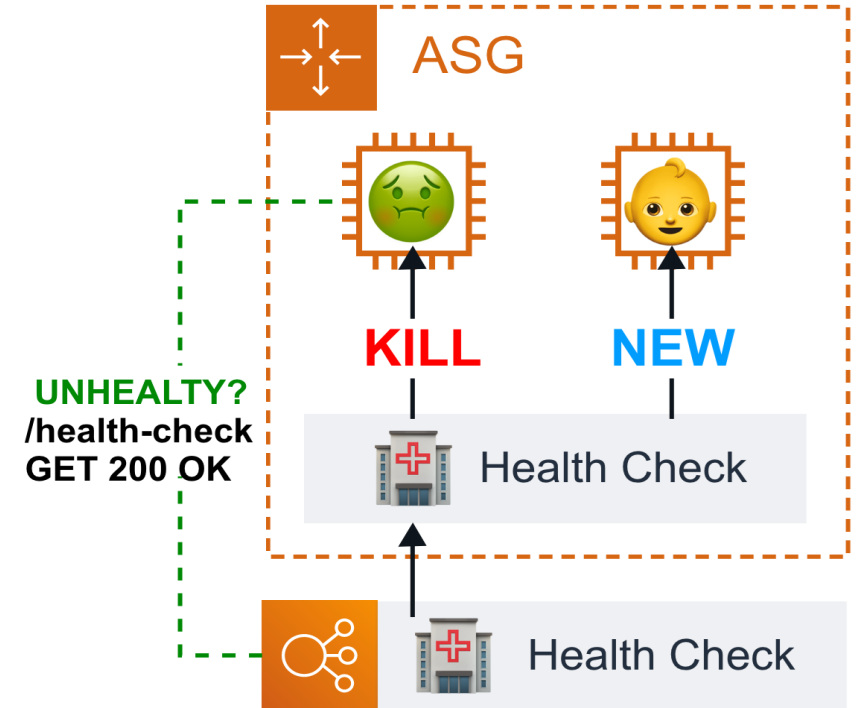
## EC2 Health Check

If the EC2 instance fails either of its EC2 Status Checks

## ELB Health Check

ASG will perform a health check based on the ELB health check.  
ELB pings an HTTP endpoint at a specific path, port and status code.

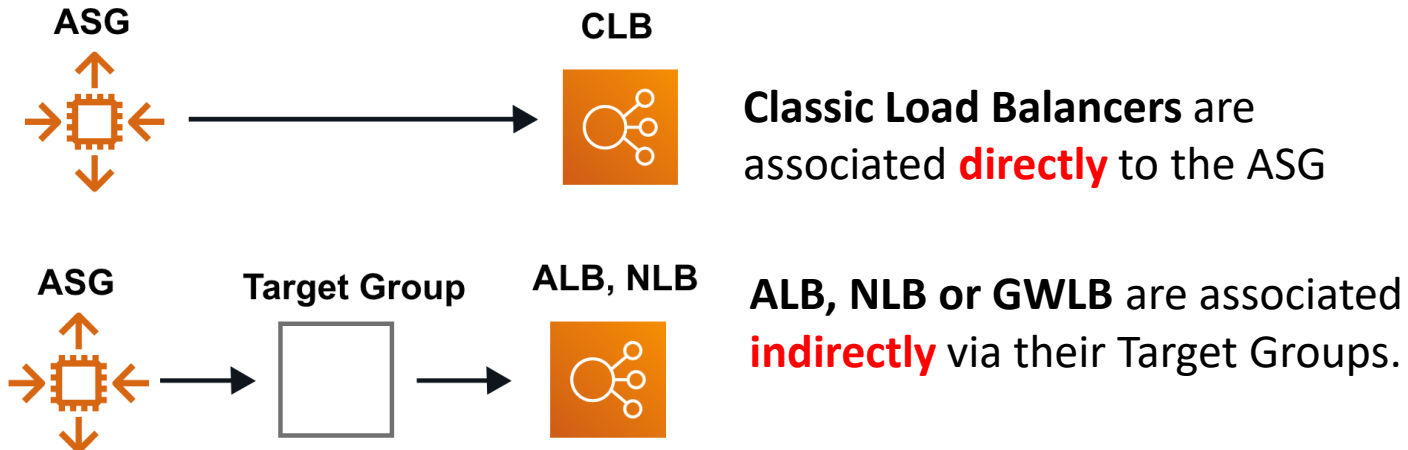
```
aws autoscaling update-auto-scaling-group \  
--auto-scaling-group-name my-asg \  
--health-check-type ELB \  
--health-check-grace-period 600 \  
--vpc-zone-identifier "subnet-5ea0c127,subnet-6194ea3b,subnet-c934b782"
```



# ASG - ELB Integration

An Elastic Load Balancer (ELB) **can be attached** to your Auto Scaling Group (ASG)

```
aws autoscaling attach-load-balancer-target-groups \  
--auto-scaling-group-name my-asg \  
--target-group-arns ...
```



An attached ELB means that the ASG can use the Load Balancer's health check instead of EC2

# ASG – Dynamic Scaling Policy

**Dynamic Scaling Policies** are how much ASG should change capacity.

Policies are triggered by based on Cloud Watch Alarms

There are 3 dynamic scaling policies:

- Simple Scaling
- Step Scaling
- Target Tracking

Adjustment Types determine how capacity should change

- **ChangeInCapacity** — change capacity based on the scaling adjustment
- **ExactCapacity** — change capacity to an exact number
- **PercentChangeInCapacity** — change capacity by a percentage

*Adjustment Types is only for Simple and Step Scaling Policies*

# ASG – Simple Scaling Policy

**Simple Scaling Policies** simply change capacity in either direction by a certain amount when a CloudWatch Alarm is triggered

Create the scale-in / scale out policy

```
# Scale Out
aws autoscaling put-scaling-policy \
--policy-name my-simple-scale-out-policy \
--auto-scaling-group-name my-asg \
--scaling-adjustment 30 \
--adjustment-type PercentChangeInCapacity
```

```
# Scale In
aws autoscaling put-scaling-policy \
--policy-name my-simple-scale-in-policy \
--auto-scaling-group-name my-asg \
--scaling-adjustment -1 \
--adjustment-type ChangeInCapacity \
--cooldown 180
```

Create the cloudwatch alarm for each policy

```
aws cloudwatch put-metric-alarm \
--alarm-name "my-asg-scale-out-alarm" \
--metric-name CPUUtilization \
--namespace AWS/EC2 \
--statistic Average \
--period 300 \
--threshold 70 \
--comparison-operator GreaterThanThreshold \
--dimensions Name=AutoScalingGroupName,Value=my-asg \
--evaluation-periods 2 \
--alarm-actions <arn-to-scale-out-policy> \
--unit Percent
```

*\*This example is not showing the CloudWatch Alarm for scale-in*



When using Simple Scaling Policy its recommended to set a cooldown period.

Its recommended to not use Simple Scacling Policy and cooldown and instead use Step or Target Tracking policies

# ASG – Step Scaling Policy

**Step Scaling Policies** change capacity in either direction by a certain amount at different thresholds (steps) when a CloudWatch Alarm is repeatedly triggered.

```
aws autoscaling put-scaling-policy \  
  --auto-scaling-group-name my-asg \  
  --policy-name my-step-scale-out-policy \  
  --policy-type StepScaling \  
  --adjustment-type PercentChangeInCapacity \  
  --metric-aggregation-type Average \  
  --step-adjustments MetricIntervalLowerBound=0.0,MetricIntervalUpperBound=15.0,ScalingAdjustment=10 \  
    MetricIntervalLowerBound=15.0,MetricIntervalUpperBound=25.0,ScalingAdjustment=20 \  
    MetricIntervalLowerBound=25.0,ScalingAdjustment=30 \  
  --min-adjustment-magnitude 1
```

*\*This example is not showing the CloudWatch Alarm for step scaling policy*

# ASG – Target Tracking Scaling Policy

**Target tracking scaling policy** automatically scales the capacity of your ASG based on a target metric value.

## Metric Types

- ASGAverageCPUUtilization
- ASGAverageNetworkIn
- ASGAverageNetworkOut
- ALBRequestCountPerTarget
- or use a Custom Metric

Target Tracking Scaling Policy **will create two CloudWatch alarms** whereas Simple and Step Scaling Policy you have to create the CloudWatch Alarms yourself (unless you're using the AWS Console which creates them for you)

```
aws autoscaling put-scaling-policy \  
--policy-name cpu50-target-tracking-scaling-policy \  
--auto-scaling-group-name my-asg \  
--policy-type TargetTrackingScaling \  
--target-tracking-configuration file://config.json
```

```
# config.json  
{  
  "TargetValue": 50.0,  
  "PredefinedMetricSpecification":  
    {  
      "PredefinedMetricType": "ASGAverageCPUUtilization"  
    }  
}
```

# ASG – Predictive Scaling Policy

**Predictive Scaling Policy** triggers scaling by **analyzing historical load data** to detect **daily** or **weekly** patterns in traffic flows

- You need a forecast 24 hour CloudWatch Data before you can create a predictive scaling policy
- Predictive Scaling Policy will continuously use the last 14 days of data to tweak the policy
- It will produce hourly forecast for capacity requirements for the next 48 hours
- It will update every 6 hours using the latest CloudWatch data

```
aws autoscaling put-scaling-policy \  
--policy-name my-policy \  
--auto-scaling-group-name my-asg \  
--policy-type PredictiveScaling \  
--predictive-scaling-configuration file://config.json
```

**Forecast Only**

```
{  
  "MetricSpecifications": [{  
    "TargetValue": 40,  
    "PredefinedMetricPairSpecification": {  
      "PredefinedMetricType": "ASGCPUtilization"  
    }  
  }],  
  "Mode": "ForecastOnly"  
}
```

**Forecast and Scale**

```
{  
  "MetricSpecifications": [{  
    "TargetValue": 40,  
    "PredefinedMetricPairSpecification": {  
      "PredefinedMetricType": "ASGCPUtilization"  
    }  
  }],  
  "Mode": "ForecastAndScale"  
}
```

# ASG –Termination Policies

**ASG Termination Policies** decide the order for terminating instances.

AWS provides **predefined policies**:

- Default
- AllocationStrategy
- OldestLaunchTemplate
- OldestLaunchConfiguration
- ClosestToNextInstanceHour
- NewestInstance
- OldestInstance



```
aws autoscaling update-auto-scaling-group \  
  --auto-scaling-group-name my-asg \  
  --termination-policies \  
    "OldestLaunchConfiguration" \  
    "ClosestToNextInstanceHour"
```

You can create a custom termination policy by invoking a Lambda Function.

# ASG – Instance Refresh

**ASG Instance refresh** allows you to **replace multiple instances** when you have **new configurations changes** you're required to rollout.

**Instance warmup** — how long to let the instance warmup in seconds

**Min/Max healthy percentages** — The percentage of healthy during rollout

**Checkpoints** — Pause the instance refresh after a certain percentage of replacements to verify progress.

**Skip matching** — Compare old instances to the new configuration and only replace those that don't match.

**Multiple instance types** — Apply a new or updated mixed instances policy as part of the desired configuration.

**Auto Rollback** — If it fails it to automatically rollback to previous version

**CloudWatch Alarm** — Notify CloudWatch

```
aws autoscaling start-instance-refresh \
--cli-input-json file://config.json

# file://config.json

{
  "AutoScalingGroupName": "my-asg",
  "Preferences": {
    "InstanceWarmup": 60,
    "MinHealthyPercentage": 50,
    "AutoRollback": true,
    "ScaleInProtectedInstances": Ignore,
    "StandbyInstances": Terminate
  }
}
```

# ASG –Instance Scale-In Protection

**Instance scale-in protection** prevents instances from being terminated on an ASG.



Scale-in protection is turned off by default on an ASG

```
aws autoscaling update-auto-scaling-group \  
--auto-scaling-group-name my-asg \  
--new-instances-protected-from-scale-in
```

You can enable or disable scale-In protect at time of creation or any other time.

# ASG – Launch Template

A Launch Template is **EC2 instance configuration information** used to launch an instance into an autoscaling group.

A Launch Template includes:

- Amazon Machine Image (AMI)
- Instance Type
- Key Pair
- Security Groups
- User Data
- Network Interfaces
- Block Device Mappings
- And other parameters commonly seen when configuring the launch of an EC2 instance



A Launch Template is required to make an ASG work.



\*You can use SSM Parameter instead of the AMI for Launch Templates so you don't have to hardcode the ImageID

The **only parameter required is the AMI** for a Launch template

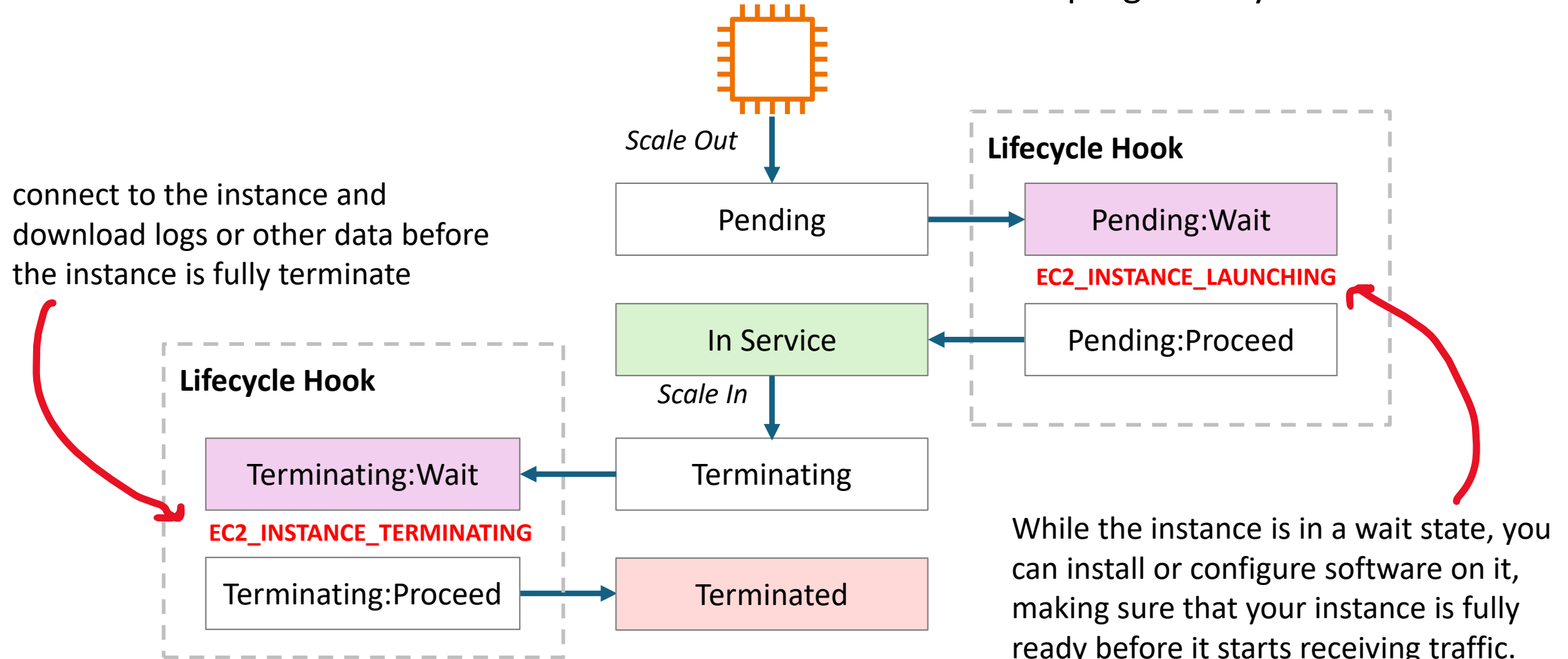
```
aws ec2 create-launch-template \  
--launch-template-name my-launch-template \  
--version-description "My first version" \  
--launch-template-data file://launch-template-data.json
```

```
{  
  "ImageId": "ami-1234567890abcdef0",  
  "InstanceType": "t2.micro",  
  "KeyName": "my-key-pair",  
  "SecurityGroupIds": ["sg-0123456789abcdef0"],  
  "IamInstanceProfile": {"Name": "my-iam-role"},  
  "BlockDeviceMappings": [  
    {  
      "DeviceName": "/dev/sda1",  
      "Ebs": {  
        "VolumeSize": 8,  
        "DeleteOnTermination": true,  
        "VolumeType": "gp2"  
      }  
    }  
  ]  
}
```

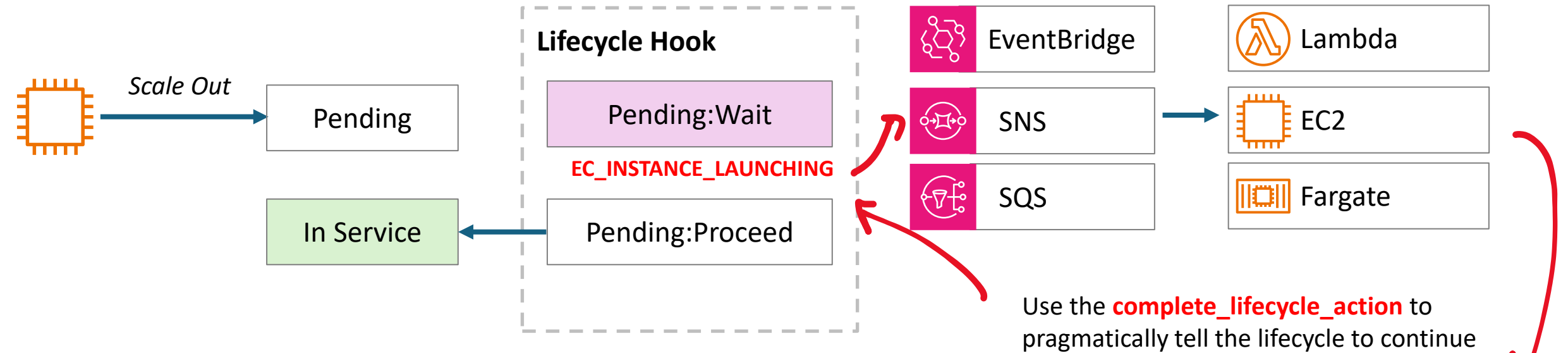
- When you create a Launch Template with the same name it will push a new version
- An ASG can be set to Launch a specific Launch Template version or the Latest version.
- **Launch Configurations** is the legacy system that are no longer used to launch instances with ASG

# ASG – Lifecycle Hooks

**Lifecycle hooks** enable you to **perform custom actions** when an instance is Pending or Terminating. You tell ASG to wait in these until it either times out or an external program says it can transition.



# ASG – Lifecycle Hooks



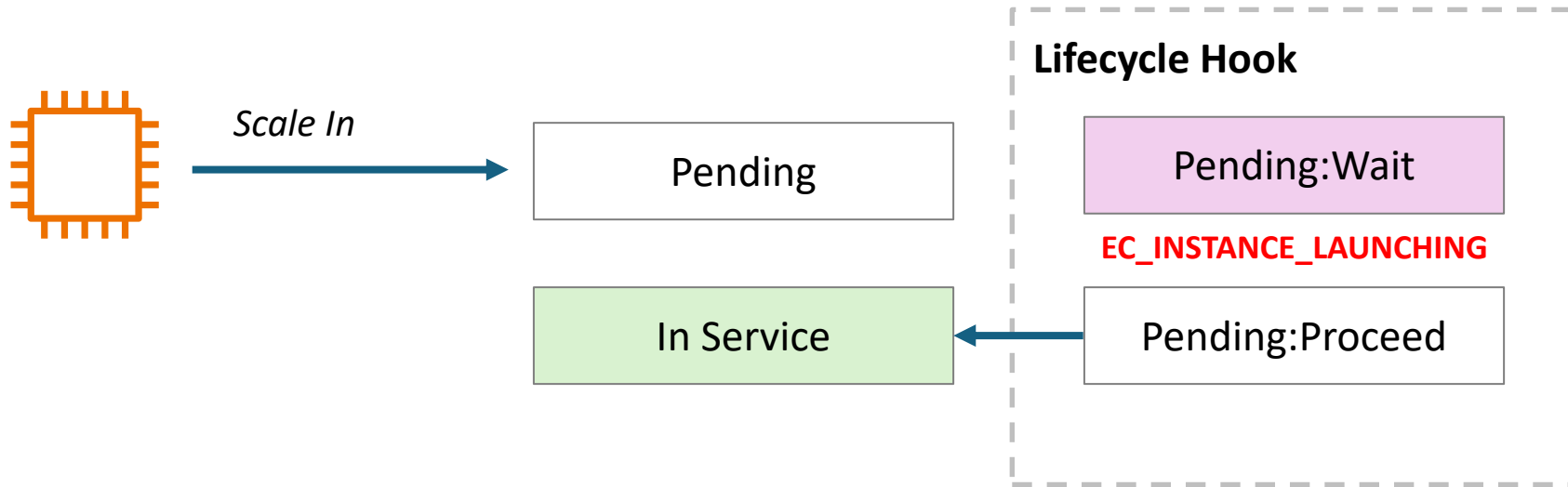
```
aws autoscaling put-lifecycle-hook \  
--auto-scaling-group-name my-asg \  
--lifecycle-hook-name my-launch-hook \  
--lifecycle-transition autoscaling:EC2_INSTANCE_LAUNCHING \  
--notification-target-arn arn:aws:sqs:ca-central-1:123456789012:my-queue \  
--role-arn arn:aws:iam::123456789012:role/my-notification-role \  
--notification-metadata "SQS message metadata" \  
--heartbeat-timeout 4800
```

You can **notify** an SQS or SNS queue, or you can **indirectly** have Amazon EventBridge rule trigger and event.

```
client.complete_lifecycle_action({  
    auto_scaling_group_name: my_asg,  
    lifecycle_hook_name: my_hook,  
    instance_id: instance_id,  
    lifecycle_action_token: token,  
    lifecycle_action_result: 'CONTINUE'  
})
```

The **heartbeat** tells ASG how long to remain in the state before timing out. The default is 3600 seconds (1 hour).

# ASG – Lifecycle Hooks



Create a notification target and an IAM role.

The target can be either an SQS queue or an SNS topic.

The role allows ASG to publish lifecycle notifications to the target.

Create a Lambda function and a rule that allows Amazon EventBridge to invoke your Lambda function when an instance is put into a wait state due to a lifecycle hook.

Create a launch template with a user data script that runs while an instance is in a wait state due to a lifecycle hook.

# ASG – Warm Pools

A warm pool is a pool of pre-initialized EC2 instances that sits alongside an ASG



A warm pool **decreases latency for your app** that have **long boot times**

- An ASG can pull from the warm pool to meet its desired capacity
- When an instance leaves the warm pool this is called a **warm start**
- The **warm pool** size is determined by the difference of Max Size (MS), Max Prepared Capacity (MPX) and Desired Capacity (DC)
- You can keep instances in the warm pool in one of three states: Stopped, Running, or Hibernated
- Lifecycle hooks are used to put an instance into a Wait state.
- Instance reuse policy can return instances back to the pool instead of being terminated

```
aws autoscaling put-warm-pool \  
--auto-scaling-group-name my-asg \  
--max-group-prepared-capacity 4 \  
--min-size 2 \  
--pool-state Running
```

You set a **Max and Min** size for your warm pool.



# ASG – Detach / Attach Instances

You can detach instances from your ASG that instance becomes independent and can either be managed on its own or attached to a different ASG

Criteria for **attaching**:

- EC2 instance is in running state
- AMI used to launch the instance exists
- Not a member of another ASG (detached)
- In one of the same AZs as the ASG
  - And the ELB is an ASG is associated with an ELB

Criteria for **detaching**:

- Only can detach when instance is InService ASG state
- If you detach an instance in StandBy AWS state this could cause other instances to terminate
- You can decrement desired capacity at the time of detachment

```
aws autoscaling attach-instances \  
--instance-ids i-05b4f7d5be44822a6 \  
--auto-scaling-group-name my-asg
```

```
aws autoscaling detach-instances \  
--instance-ids i-05b4f7d5be44822a6 \  
--auto-scaling-group-name my-asg
```

```
aws autoscaling detach-instances \  
--instance-ids i-05b4f7d5be44822a6 \  
--auto-scaling-group-name my-asg \  
--should-decrement-desired-capacity
```

# ASG – Temporarily Remove Instances

You can put an instance that is in the InService state into the Standby state, update or troubleshoot the instance, and then return the instance to service.

Instances that are on standby are still part of the Auto Scaling group, but they do not actively handle load balancer traffic.

Enter standby

```
aws autoscaling enter-standby \  
--instance-ids i-05b4f7d5be44822a6 \  
--auto-scaling-group-name my-asg \  
--should-decrement-desired-capacity
```

Exit standby

```
aws autoscaling exit-standby \  
--instance-ids i-05b4f7d5be44822a6 \  
--auto-scaling-group-name my-asg
```

# ASG – Suspend / Resume Processes

Suspending processes can be useful when you need to investigate or troubleshoot an issue without interference from scaling policies or scheduled actions.

The suspend-resume feature supports the following processes:

- Launch
- Terminate
- AddToLoadBalancer
- AlarmNotification
- AZRebalance
- HealthCheck
- InstanceRefresh
- ReplaceUnhealthy
- ScheduledActions

Suspending a a specific process

```
aws autoscaling suspend-processes \  
--auto-scaling-group-name my-asg \  
--scaling-processes HealthCheck ReplaceUnhealthy
```

Suspending all processes

```
aws autoscaling suspend-processes \  
--auto-scaling-group-name my-asg
```

# ASG – Troubleshooting

An Auto Scaling Group can fail for multiple reasons:

## EC2 Instance Launch Failures

- ASG not found
- AZ no longer supported
- Invalid block device
- Instance Type not supported in AZ
- Key pair does not exist
- Launch Configuration not supported
- Placement Group not allowed be to used by Instance Types
- Security Group does not exist

## AMI Issues

- AMI ID does not exist
- AMI is pending and cannot be run
- virtualName is invalid
- Architecture mismatch

## Load Balancer Issues

- Cannot find load balancer
- Instances is not in VPC
- No active load balancer
- The security token is invalid

## Capacity Limits

- Maximum number of instances are already running
- You've reached your capacity allowed for the AZ

To find out the exact error use the following AWS CLI command



```
aws autoscaling describe-scaling-activities \  
--auto-scaling-group-name my-asg
```